

鱼与熊掌可以兼得的机器学习

李宏毅《机器学习》2025 · 为什么深度学习有潜力



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 42 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=11>

目录

1	从 P08 的两难出发	3
2	复习：一个 Hidden Layer 已经能逼近任何函数	4
3	实验观察：深一点通常比只变胖更有效	6
4	为什么深层结构更有效？	7
4.1	逻辑电路类比	8
4.2	程序设计类比	9
4.3	剪窗花类比	9
5	一个 ReLU 构造：深层网络如何产生指数多片段	10
6	复杂但有规律：哪些任务适合深度学习？	12
7	本讲综合：深度学习如何让鱼与熊掌兼得	13

1 从 P08 的两难出发

P08 的结尾留下一个两难。若模型家族 $\mathcal{H}$ 很大，理想函数

$$h^{all} = \arg \min_{h \in \mathcal{H}} L(h, \mathcal{D}_{all})$$

有机会把理想 loss 压得很低；但 $\mathcal{H}$ 大也会增加训练集与全体资料落差的风险。若 $\mathcal{H}$ 很小，现实比较容易接近理想；但理想本身可能很差，因为候选函数太少。

这一讲要回答的问题是：有没有办法让理想 loss 低，同时又让模型复杂度不至于太大？这就是课堂说的「鱼与熊掌可以兼得」。

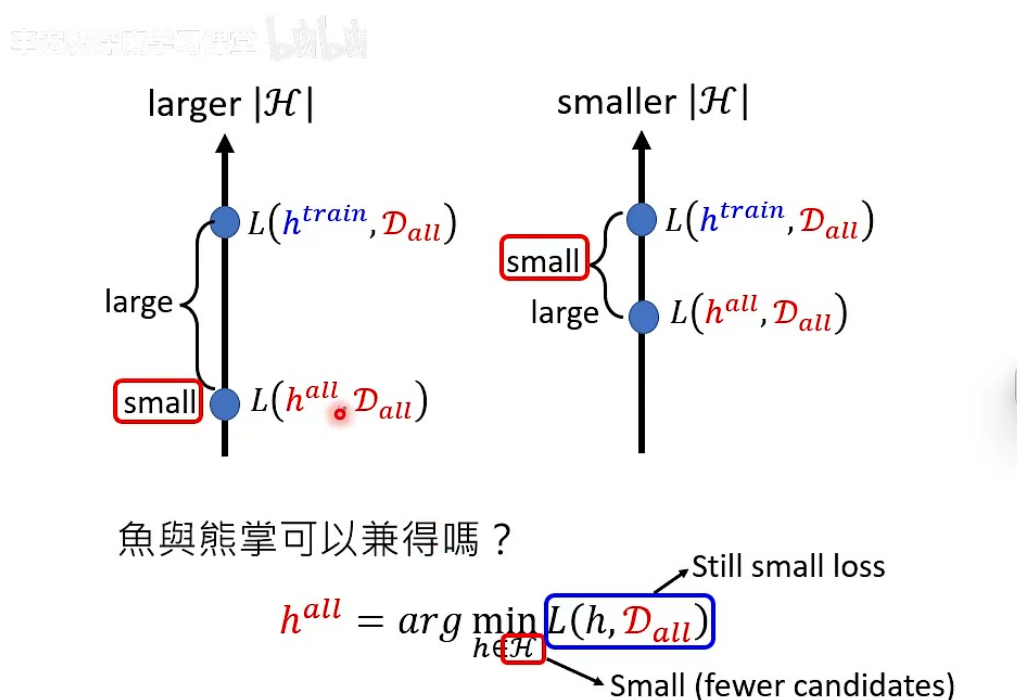


图 1: P08 的复杂度两难：大 $\mathcal{H}$ 让理想好但现实可能远；小 $\mathcal{H}$ 让现实近但理想可能差。目标是找到成员少、但成员足够好的 $\mathcal{H}$ 。画面依据：约 00:00–02:25。

课堂把理想状况说成：找到一个神奇的 $\mathcal{H}$ ，它一方面候选函数不多，因此泛化风险较低；另一方面候选函数都是「精英」，能让 $L(h^{all}, \mathcal{D}_{all})$ 仍然很小。深度学习的潜力就在于，它可能用结构化的方式做到这件事。

这里的核心不是“模型越大越好”

如果只是把模型做得很大，当然可能让理想 loss 变低，但也会提高过拟合风险。深度学习真正有趣的地方是：对某些复杂但有规律的目标函数，深层结构能用比浅层结构少得多的参数表达它们。

2 复习：一个 Hidden Layer 已经能逼近任何函数

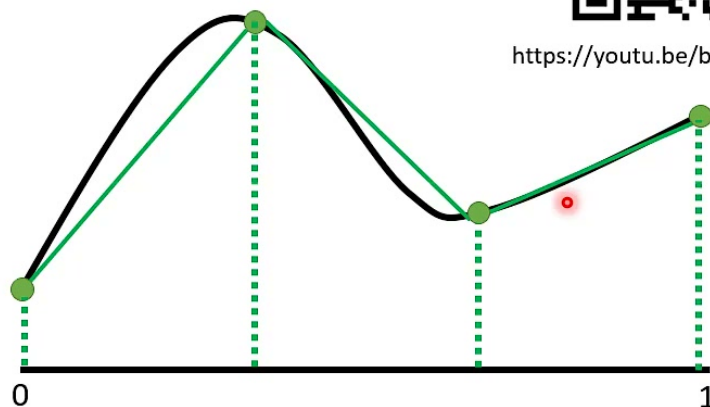
在讲深度的好处前，老师先复习为什么 hidden layer 有用。考虑一维输入 x 与输出 y ，如果我们想逼近任意连续曲线，可以先用 piecewise linear function 近似它。切得越细，折线越接近原函数。

手机软件应用学习课堂 bilibili

Piecewise Linear



<https://youtu.be/bHcJcP2Fyxs>



We can have good approximation with sufficient pieces.

图 2: 任意曲线可以用足够多段 piecewise linear function 近似。画面依据：约 03:20–04:35。

Piecewise linear function 又可以看成常数项加上一组阶梯状函数或折线函数。若 activation 是 sigmoid，可以用很陡的 sigmoid 近似 hard sigmoid；若 activation 是 ReLU，也可以用两个 ReLU 组合出类似 hard sigmoid 的形状。

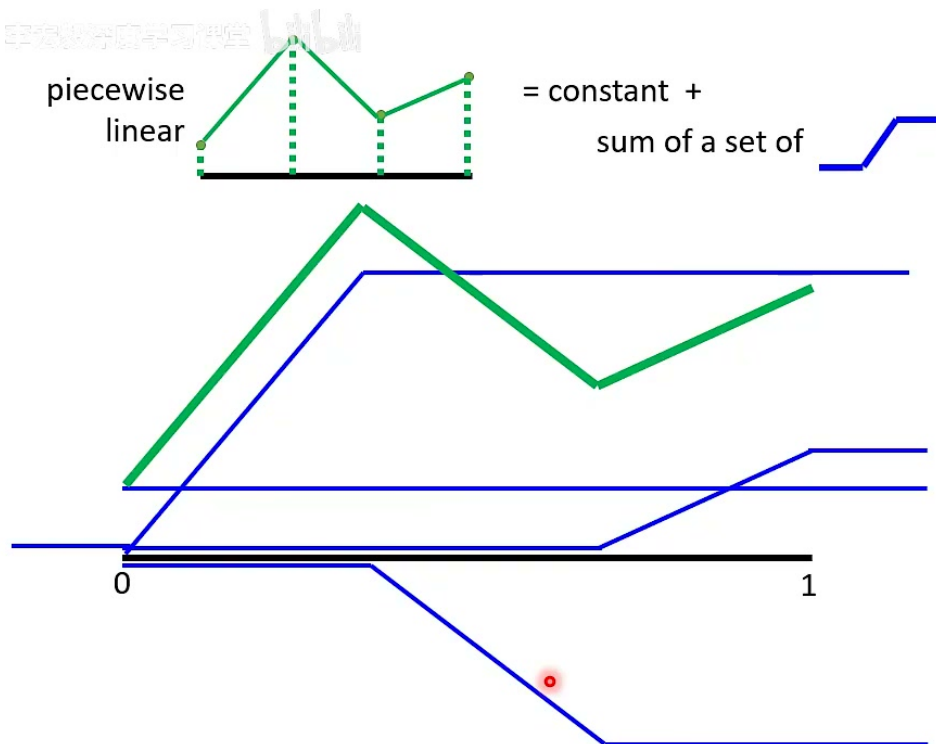


图 3: Piecewise linear function 可分解成常数项加上一组阶梯或折线形函数。画面依据：约 04:50–06:15。

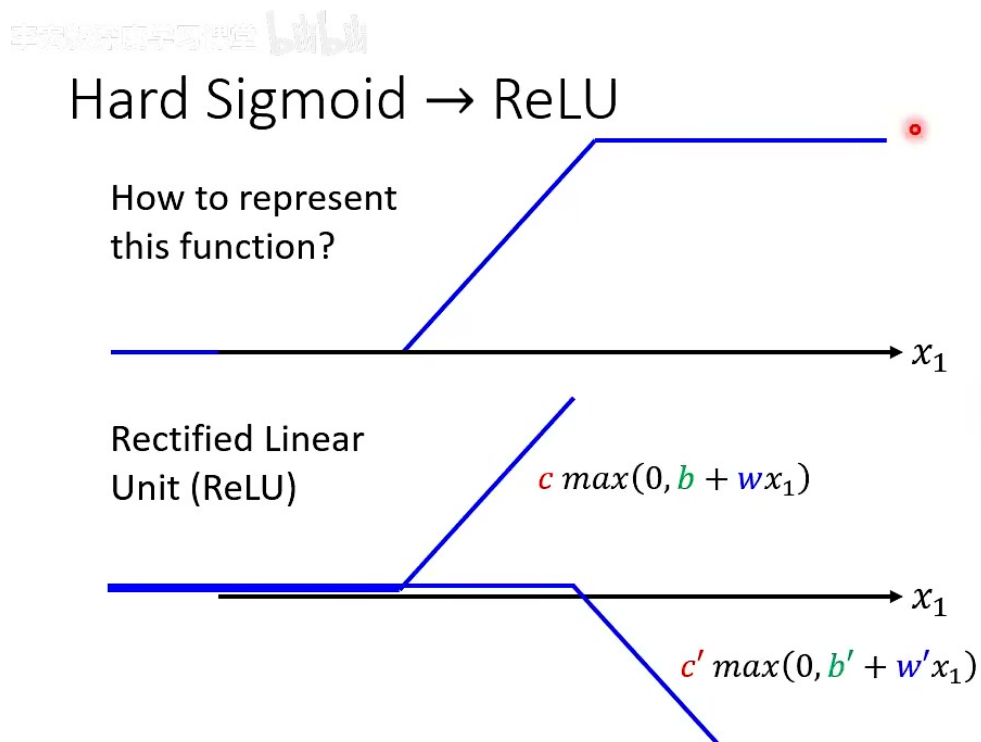


图 4: 两个 ReLU 可组合出 hard sigmoid 形状；足够多 ReLU 可组合出足够多折线段。画面依据：约 07:30–08:40。

因此，一个足够宽的一层 hidden network 可以逼近任意函数。这就是 universal approximation 的直觉版本。

那为什么还要 deep ?

如果一个 hidden layer 已经能表示任何函数，问题就不再是“能不能表示”，而是“用多少参数表示”。深度学习的优势不是把不可表示的函数变成可表示，而是对很多有结构的函数，用更少参数、更有效率地表示。

3 实验观察：深一点通常比只变胖更有效

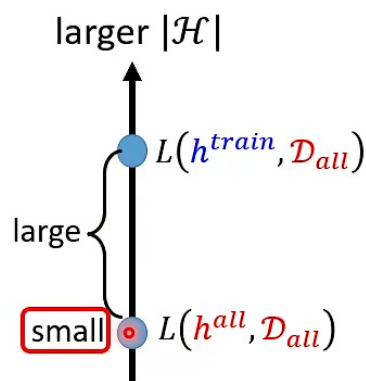
课堂引用了一篇语音辨识论文的结果。随着层数增加，word error rate 下降：

Layer $\times$ Size	Word Error Rate (%)
$1 \times 2k$	24.2
$2 \times 2k$	20.4
$3 \times 2k$	18.4
$4 \times 2k$	17.8
$5 \times 2k$	17.2
$7 \times 2k$	17.1

深度学习应用案例

Deeper is Better?

Layer X Size	Word Error Rate (%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1



Seide Frank, Gang Li, and Dong Yu. "Conversational Speech Transcription Using Context-Dependent Deep Neural Networks." *Interspeech*. 2011.

图 5: 语音辨识实验中，层数加深后 word error rate 下降；但这还不能证明深度本身的价值，因为参数量也增加了。画面依据：约 13:30–14:55。

有人会质疑：深层网络表现好，只是因为参数更多、模型更大。若资料足够多，大模型当然能让理想更好；这不说明「深」特别重要。为了回答这个质疑，需要比较参数量相近的 shallow/fat network 与 deep/thin network。



Fat + Short v.s. Thin + Tall

Layer X Size	Word Error Rate (%)	Layer X Size	Word Error Rate (%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

Seide Frank, Gang Li, and Dong Yu. "Conversational Speech Transcription Using Context-Dependent Deep Neural Networks." *Interspeech*. 2011.

图 6: Fat + Short vs. Thin + Tall: 参数量相近时，把网络变深通常比单层变胖表现更好；一层 16k neuron 也不如较小的两层网络。画面依据：约 16:15–18:05。

实验给出的启发是：同样参数预算下，把参数沿深度方向组织起来，可能比全部塞进一层更有效。接下来要解释为什么。

不要把 deep learning 理解成“参数更多”

很多人把深度学习和大模型、大资料、容易 overfit 画等号。课堂要反过来强调：在某些目标函数上，deep network 可能用更少参数完成同样表达，因此反而更不容易 overfit，或需要更少训练资料。

4 为什么深层结构更有效？

结论先说：若目标函数是 **复杂但有规律的**，deep network 往往优于 shallow network。复杂代表函数需要很多局部变化；有规律代表这些变化不是完全随机，而是可以通过组合结构重复利用。

Why we need deep?

Yes, one hidden layer can represent any function.

However, using deep structure is more effective.

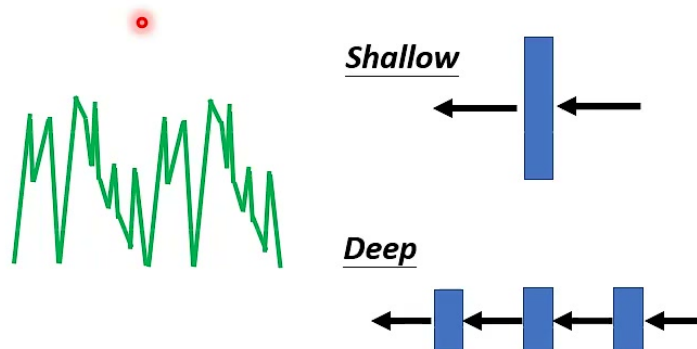


图 7: 一层 hidden layer 可以表示任意函数，但深层结构通常更有效率。画面依据：约 18:20–20:10。

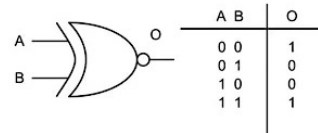
课堂用三个类比说明这种结构化效率。

4.1 逻辑电路类比

Parity check 的任务是判断输入 bit 序列里 1 的个数是奇数还是偶数。理论上，两层逻辑电路可以表示任何布尔函数，但如果强行用两层来做 parity check，可能需要 2^d 级别的 gates，其中 d 是输入长度。

更自然的做法是把 XNOR gates 串起来：前两个 bit 先合并，再和第三个合并，再和第四个合并。这样形成一个深的结构，使用的 gates 数量远少于两层暴力表示。

Analogy – Logic Circuits



- E.g., parity check



图 8: 逻辑电路类比: 两层电路可表示任何布尔函数, 但 parity check 用串接结构更有效率。画面依据: 约 20:20–23:35。

4.2 程序设计类比

写程序时, 我们不会把所有逻辑塞进 main function, 而会拆成 function、module、subroutine。这样同一功能可以反复调用, 程序更短、更可读, 也更容易维护。

神经网络中的深层结构也有类似意味: 后层可以复用前层已经算出的中间表示, 而不是每次从原始输入重新组合一切。

4.3 剪窗花类比

剪窗花时, 直接在展开的纸上剪复杂图案很麻烦; 先折纸, 再剪几刀, 展开后就得到复杂对称图案。折纸过程让简单操作通过结构重复产生复杂结果。

More Analogy

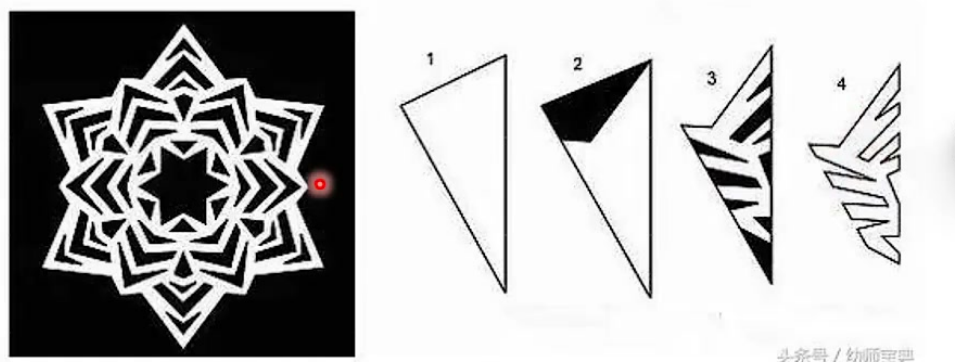


图 9: 剪窗花类比: 折叠结构让少量剪刀动作产生复杂图案; 深层网络也通过组合结构高效表达复杂函数。画面依据: 约 24:45-25:45。

共同点

逻辑电路、程序模块、剪窗花的共同点是: 有结构的组合能复用中间结果。Deep network 不是把参数随便堆高, 而是让前层输出成为后层输入, 让简单变换反复组合, 产生指数级增长的表达片段。

5 一个 ReLU 构造: 深层网络如何产生指数多片段

课堂接着用一个一维 ReLU 网络展示深层结构的效率。先看一层网络, 输入 x , 两个 ReLU neuron:

$$a_1 = \text{ReLU}(x - 0.5) + \text{ReLU}(-x + 0.5).$$

它产生一个 V 字形关系。接第二层时, 用同样结构作用在 a_1 上, 得到 a_2 ; 由于 a_1 本身已经是 V 字形, 第二层会把它折成更多段, 形成 W 字形。

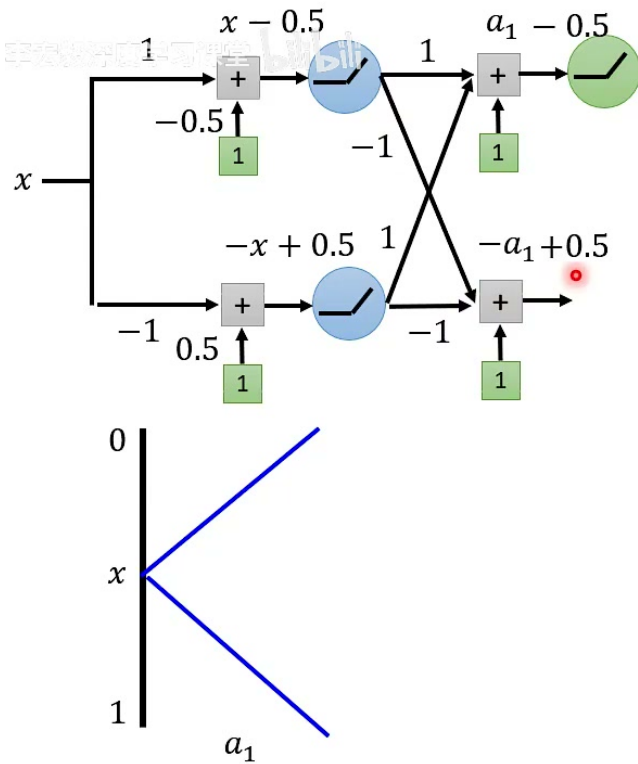


图 10: 两层 ReLU 结构把一层的 V 字形继续折叠, 产生更多 piece。画面依据: 约 29:50–33:05。

继续叠第三层, 会得到 2^3 个 pieces。一般地, 若有 K 层, 每层只放 2 个 ReLU neuron, 总共 $2K$ 个 neuron, 就能产生 2^K 个线段片段。

若用 shallow network 做同样的锯齿状函数, 一个 ReLU neuron 大致只能贡献一个新折点或一段变化; 要产生 2^K 个 pieces, 就需要 2^K 级别的 neuron。于是 deep 与 shallow 在参数需求上出现指数差距:

$$\text{deep: } O(K) \quad \text{shallow: } O(2^K).$$

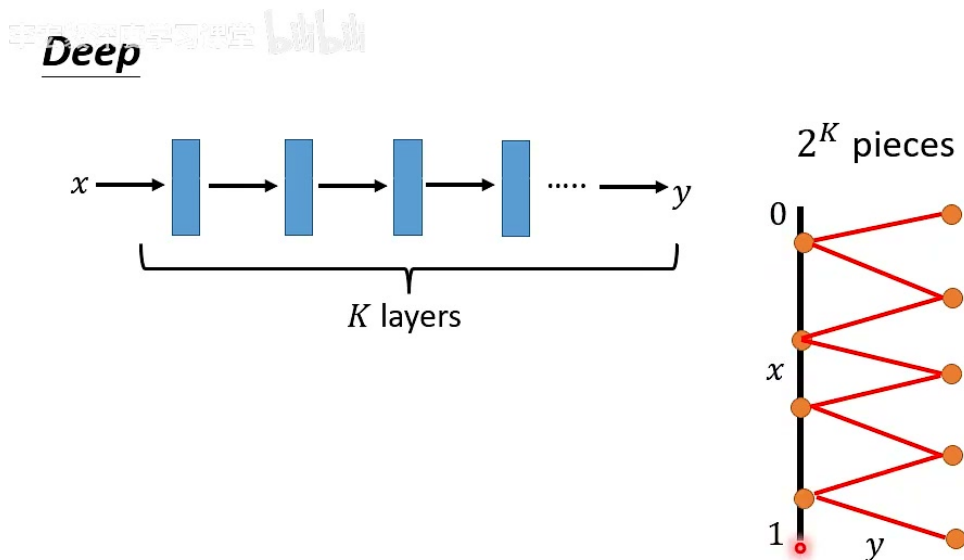


图 11: Deep network 用 K 层、每层 2 个 neuron 产生 2^K pieces; shallow network 若要产生同样 pieces, 需指数多 neuron。画面依据: 约 36:35–38:15。

为什么这能回应“鱼与熊掌”？

如果目标函数需要很多 pieces, 但这些 pieces 有规律, deep network 可以用少量参数表达它。少量参数意味着候选函数集合更受限制, 泛化风险较低; 同时表达能力又足以让理想 loss 很低。因此深层结构有机会同时取得低理想 loss 与较小泛化落差。

6 复杂但有规律：哪些任务适合深度学习？

深度的优势不是对所有函数都成立。如果目标函数完全随机, 没有可复用结构, 深层网络也未必占优。课堂强调的条件是：**复杂而有规律**。

语音、影像等任务很可能满足这个条件。它们的输入空间非常复杂, 但其中有大量可组合结构：

- 影像中，边缘组成纹理，纹理组成部件，部件组成物件。
- 语音中，短时声学特征组成音素，音素组成词，词组成句子。
- 语言中，字符或 token 组成词，词组成短语，短语组成语义结构。

这些层级结构正适合 deep network：低层提取局部简单结构，高层组合成更抽象的表示。

“深”不是魔法

深层结构只是在某些有组合规律的函数上更有效率。若任务不具备这种结构，或训练方式、资料量、优化方法不合适，深层网络仍然可能失败。深度学习强，是因为它的结构偏置常常贴合真实世界资料，而不是因为层数越多一定越好。

7 本讲综合：深度学习如何让鱼与熊掌兼得

课堂最后给出结论：

深度学习是一种让鱼与熊掌兼得的方法

深度學習是一個讓
魚與熊掌可以兼得的方法

$$h^{all} = \arg \min_{h \in \mathcal{H}} L(h, \mathcal{D}_{all})$$

Still small loss

Small (fewer candidates)

图 12: 结论：深度学习是一种让鱼与熊掌可以兼得的方法，有机会用较少参数得到较低 loss。画面依据：约 41:05–41:28。

把这一讲和 P08 连起来，结论可以写成：

$$L(h^{train}, \mathcal{D}_{all}) = L(h^{all}, \mathcal{D}_{all}) + [L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all})].$$

第一项要靠表达能力压低；第二项要靠较小有效复杂度、足够资料和适当训练机制控制。浅层网络虽然理论上能表示任意函数，但表示某些复杂规律时可能需要指数多参数；参数一多，泛化就困难。深层网络通过组合结构，用较少参数表达同样复杂度的函数，因此有机会让两项同时变小。

一句话总结

深度学习的潜力不是“把模型做大”，而是“把模型做成深的组合结构”。当目标函数复杂但有规律时，深层结构能用更少参数表达它，于是既能让理想 loss 低，又能减轻过拟合压力。这就是这讲所说的鱼与熊掌可以兼得。